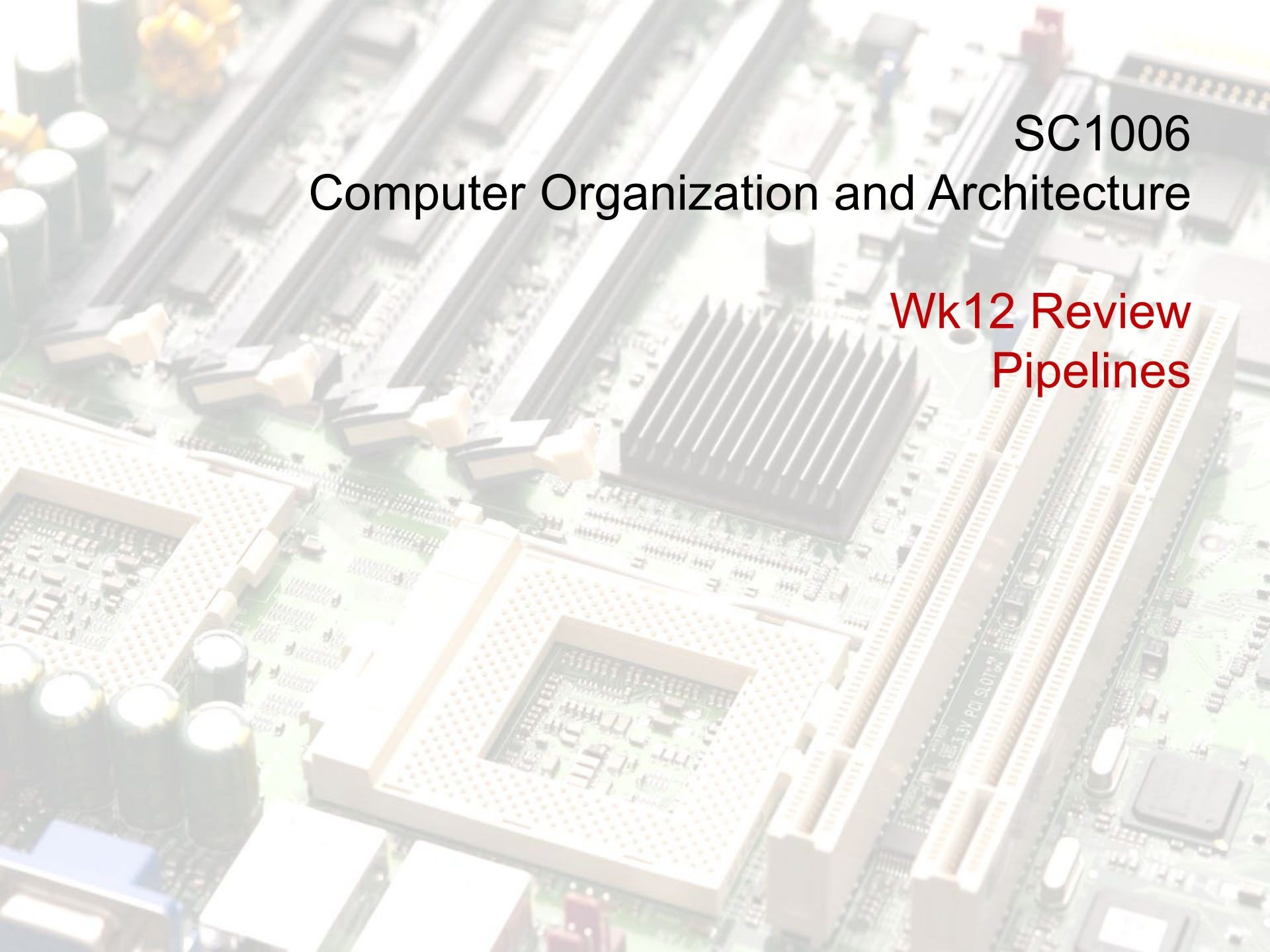




SC1006

Computer Organization and Architecture

Wk12 Review Pipelines

Discussion of important/interested issues for
pipelining...

A CPU with 5-stage instructions

No pipeline stalls

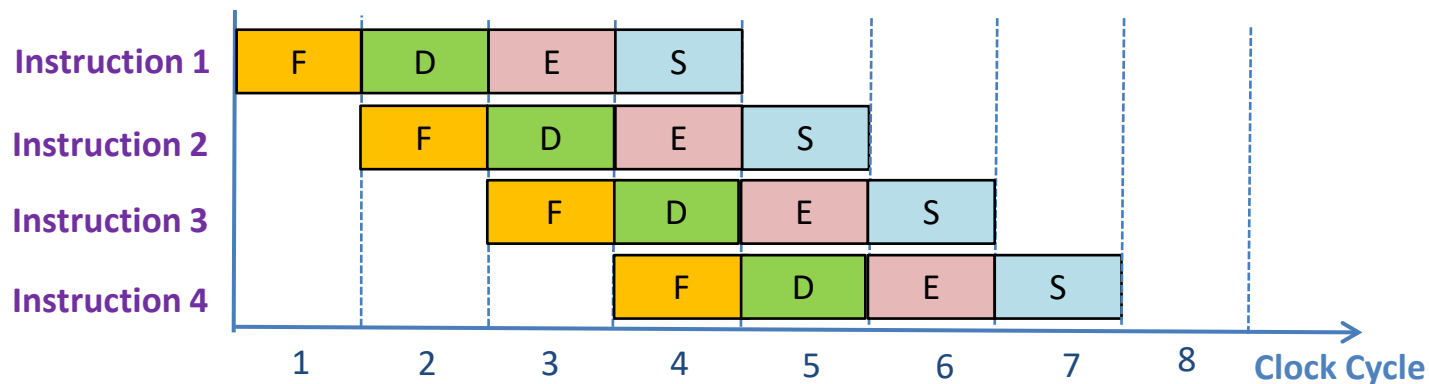
How many clock cycles to complete 10 instructions?

A CPU with 5-stage instructions

No pipeline stalls

How many clock cycles to complete 10 instructions

See 4-stage case:



$$4 + (4 - 1) = 7$$

For 10 5-stage instructions: $5 + (10 - 1) = 14$

Which of the following statement(s) are false for processor pipeline?

- A. Pipeline allows different stages of different instructions to be executed in parallel
- B. Pipeline requires independent resources to operate each of the pipeline stages to yield good efficiency
-  C. Pipeline allows different stages of the same instruction to be executed in parallel
- D. Pipeline's efficiency is maximised if there are minimum conflicts during execution

Pipeline Conflicts

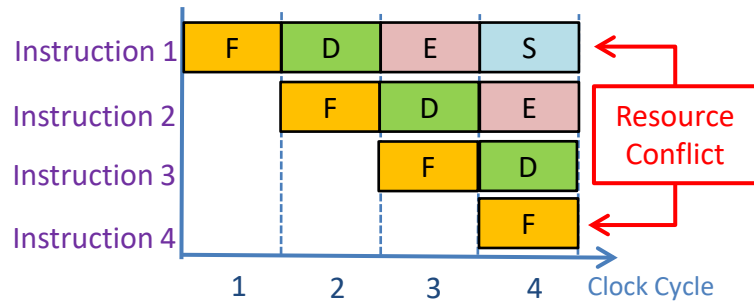
- Insufficient Resource
- Data Dependency between instructions
- Pipeline flushing due to Branch Instruction

Pipeline Conflicts

- Insufficient Resource
- Data Dependency between instructions
- Pipeline flushing due to Branch Instruction

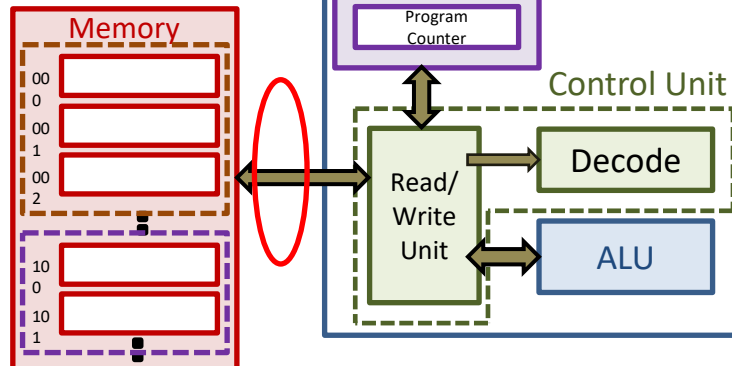
Resource Conflict

- Consider a processor with 4 pipeline stages: Fetch Instruction (F), Decode (D), Execute (E), Store Result (S)



Example: Instruction 1 is storing result in memory when instruction 4 needs to fetch a new instruction

- Resource conflict occurs when two instructions attempt to access the same resource in the same cycle. E.g., “S” is to store result to main memory (instead of a register).



Resolving Resource Conflicts

to have **multiple resources**: separate/dedicated buses (data, instruction, peripherals), more control and processing units, etc to allow simultaneous operations in any instance.

Insufficient resources to execute each of the pipeline stages will lead to

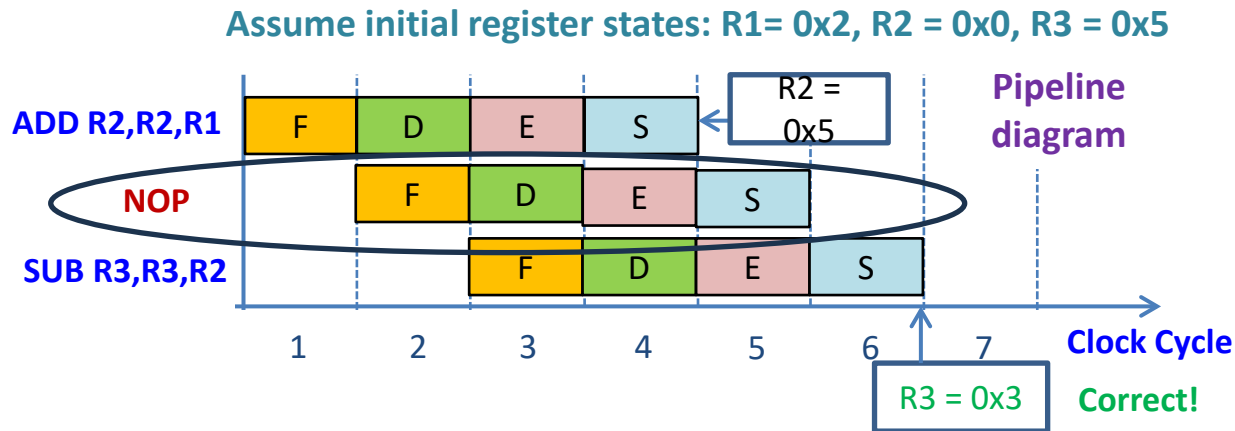
- ✓ A. Decrease in pipeline efficiency
- ✓ B. Resource conflicts
- C. Increase in pipeline efficiency
- D. Slower processor speed

Pipeline Conflicts

- Insufficient Resource
- Data Dependency between instructions
- Pipeline flushing due to Branch Instruction

Resolving Data Conflict – Insert NOP Instructions

- Compiler can analyze and insert redundant instructions to reduce data conflict
 - Data dependencies are evident in instructions during compilation
 - Compiler inserts explicit **NOP** (No Operation) instructions between instructions with data dependencies
- Delay ensures new value is available in register but causes total execution time to increase



What is the effect of data dependency conflict?

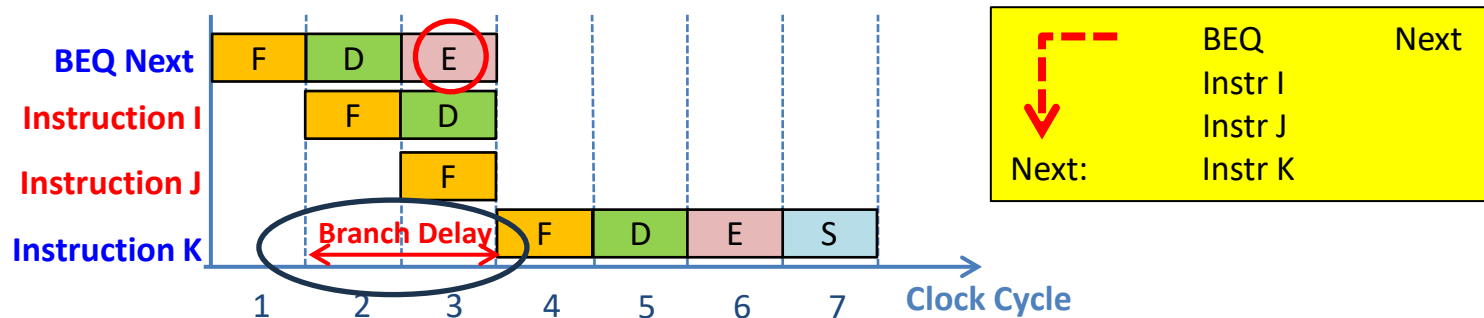
- A. Data in one register is corrupted by data in another register
- ✓ B. Data in register/memory is not updated in time for its usage.
- ✓ C. Incorrect results obtained from executing affected instructions
- D. Data in register gets modified half way through instruction execution

Pipeline Conflicts

- Insufficient Resource
- Data Dependency between instructions
- Pipeline flushing due to Branch Instruction

Branch Delay

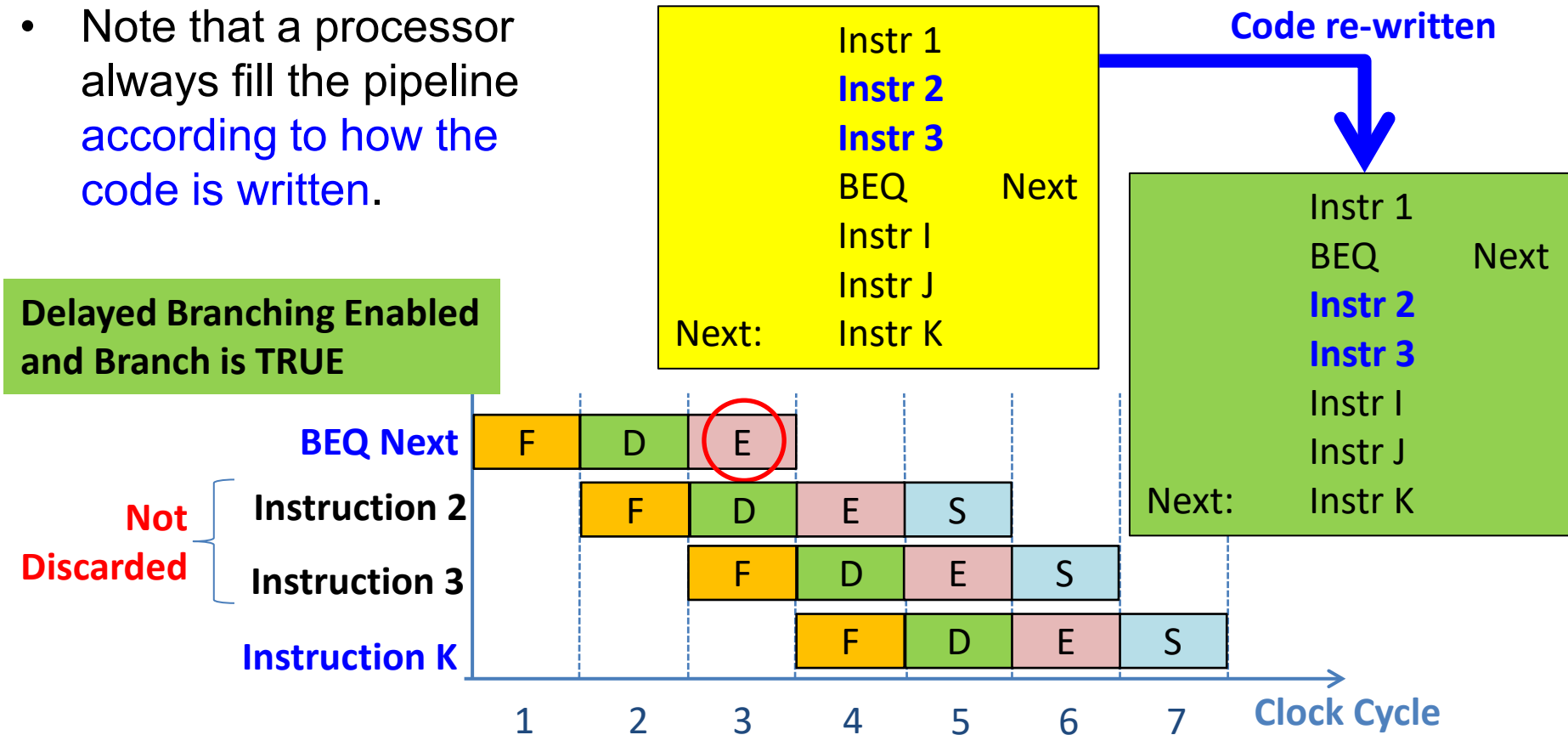
- Branch statements in pipelining can lead to **Branch Delay**.



- The **branch target** is only known after the **Execute** stage, but by this time, **Instructions I and J** have already been **fetched**.
- Instructions I and J will be **discarded**, resulting in **two-cycle branch delay**.
- The two slots that are discarded is known as the **Delay Slots**
- Instructions I and J are known as the **Delay Slot Instructions**

Delayed Branching

- The user or compiler needs to **fill the delay slots with Independent instructions**.
 - For example, if **instruction 2 and 3 are independent instructions**, they can be made to occupy the delay slots by re-writing the program code.
 - Note that a processor always fill the pipeline **according to how the code is written**.
-
- The diagram illustrates the process of re-writing code to utilize delay slots. On the left, a yellow box represents the original code sequence: 'Instr 1', 'Instr 2', 'Instr 3', 'BEQ', and 'Next'. A blue arrow labeled 'Code re-written' points from this box to a green box on the right, which contains 'Instr 1'. This represents the re-written code where independent instructions are placed in the delay slots.



Alternative solution: **dynamic branch prediction**

How would the processor behave if it predicts that the branch would be taken?

- A. Flush the pipeline early to prepare for the jump instruction
-  B. Load the instruction at the jump target instead of the next instruction in the code
- C. Load instructions into the delay slots
- D. Discard the instructions after the branch

Which statement below is FALSE?

- A. Delayed branching utilizes delay slots to execute code.
- B. In CPU that supports delayed branching, instructions in delayed slots are not flushed away.
- ✓ C. There are no restrictions to the type of instructions that can be inserted in the delay slot.
- D. The delay slot instruction is executed after the branch decision has been made.



What should the programmer do if he can't find sufficient instructions to be placed in the delay slots?

- ✓ A. Patch with NOP instructions
- B. Do nothing and let the processor hardware handle it
- C. Patch with same instruction used in the other delay slot
- D. Patch with the same Branch Instruction

Will the delay slot instructions be able to influence the branch decision?

- A. Yes
-  B. No
- C. Depend on the instructions placed in the delay slots

Please take note

The last session of Part 2

Review & More Exercise

next Mon, 11 Nov

END